Motion Planning for Self-Driving Cars

Module 1
Welcome to Course 4: Motion Planning for Self-Driving Cars

Module 2
Module 1: The Planning Problem

Module 3
Module 2: Mapping for Planning

Module 4
Module 3: Mission Planning in Driving Environments

Module 5
Module 4: Dynamic Object Interactions

Module 6
Module 5: Principles of Behaviour Planning

Module 7
Module 6: Reactive Planning in Static Environments

Module 8
Module 7: Putting it all together - Smooth Local Planning

Smooth Local Planning

Lesson 1: Parametric Curves
Video • 11 min

Lesson 2: Path Planning Optimization
Video • 12 min

Lesson 3: Optimization in Python
Video • 5 min

Lesson 4: Conformal Lattice Planning
Video • 10 min

Lesson 5: Velocity Profile Generation
Video • 12 min

Module 7 Supplementary Reading
Reading • 1h

Final Project

Final Project Overview
Video • 4 min

CARLA Installation Guide
Reading • 45 min

Course 4 Final Project
Programming Assignment • 8h

Final Project Solution [LOCKED]
Video • 7 min

Congratulations!

Congratulations for completing the course!
Video • 2 min

Congratulations on Completing the Specialization!
Video • 3 min

Programming Assignment: Course 4 Final Project

You have not submitted. You must earn 2/2 points to pass.

Deadline Pass this assignment by Mar 20, 11:59 PM PDT

Instructions My submissions Discussions

Course 4 Final Project

In this project we will be implementing many of the behavioral and local planning concepts discussed in Course 4. The goal of this project will be to have a functional motion planning stack that can avoid both static and dynamic obstacles while tracking the center line of a lane, while also handling stop signs. To accomplish this, you will have to implement behavioral planning logic, as well as static collision checking, path selection, and velocity profile generation.

To do this assignment, the CARLA simulator along with the assignment code needs to be installed. Please follow these instructions:

Install the CARLA simulator

Follow the CARLA Installation guide from a previous reading to install the CARLA simulator.

Download the final project file

Download the following "Course4FinalProject.zip" file and unpack into the subfolder folder "PythonClient" inside the "CarlaSimulator" (root) folder. This will create a subfolder "Course4FinalProject" under "PythonClient" which contains the assignment files.

Course4FinalProject
ZIP File

It is very important to have the contents of "Course4FinalProject.zip" be under the folder "**PythonClient\Course4FinalProject**" (for Windows) or "**PythonClient/Course4FinalProject**" (for Ubuntu). Installing it into another directory might cause runtime issues.

After successfully downloading CARLA and the assessment script, you can now begin the assignment.

Implementing the Motion Planner

In this project, you will be editing the "**behavioural_planner.py**", "**collision_checker.py**", "**local_planner.py**", "**path_optimizer.py**", and "**velocity_planner.py**" class files (found inside the "**PythonClient\Course4FinalProject**" (for Windows) or "**PythonClient/Course4FinalProject**" (for Ubuntu) folder). This is where you will implement your motion planner. There are 5 main aspects of the planner you will need to implement, behaviour planning logic, path generation, static collision checking, path selection, and velocity profile generation. Let's go over each of these in kind.

Behaviour Planning Logic

In this part of the project, you will implement the behavioural logic required to handle a stop sign. As in the lecture videos, you will be implementing a state machine that transitions between lane following, deceleration to the stop sign, staying stopped, and back to lane following, when it encounters a stop sign. All of the code for the behavioural planner is contained in behavioural_planner.py.

To do this, you will first implement the helper functions get_closest_index() and get_goal_index(). These will let the behavioural planner know where it is relative to the global path, and to compute the current goal point from the global path. Once these are done, you will then implement the transition_state() function, which contains the behavioural state machine logic. The required details about each of these functions are given in the code comments. Please complete all TODOs for this assignment.

Path Generation

For the path generation section of the project, the majority of the mathematical code for generating spiral paths is given to you. However, you will need to compute the goal state set (the set of goal points to plan paths to before path selection) using the get_goal_state_set() function, as well as some of the path generation helper functions. In particular, you will implement thetaf(), which computes the yaw of the car at a set of arc length points for a given spiral, optimize_spiral(), which sets up the optimization problem for a given path. Finally, once the optimization is complete, the resulting spiral will be sampled to generate the path. This functionality needs to be implemented in sample_spiral(). The required details about each of these functions are given in the code comments in the files local_planner.py and path_optimizer.py. Please complete all TODOs for this assignment.

Static Collision Checking

For this part of the motion planner, we will be editing collision_checker.py. In particular, we're going to be implementing circle-based collision checking on our computed path set using the collision_check() function. You will implement the circle location calculation for each point on the path. For futher details, please refer to the given code comments. Please complete all TODOs for this assignment.

Path Selection

The path selection portion of the project involves you evaluating an objective function over the generated path set to select the best path. The goal of this section is to eliminate paths that are in collision with static obstacles, and to select paths that both track the centerline of the global path. To encourage robust obstacle avoidance, we will also need to add a term that penalizes how close the planned path is to other paths in the path set that are in collision with a static obstacle. You will implement path selection in the select_best_path_index() function within collision_checker.py. For further details, please refer to the given code comments. Please complete all TODOs for this assignment.

Velocity Profile Generation

The last step of the project is velocity profile generation. This velocity planner will not handle all edge cases, but will handle stop signs, lead dynamic obstacles, as well as nominal lane maintenance. This is all captured in the compute_velocity_profile() function in velocity_planner.py. You will be implementing the physics functions at the end of the file which will be used for velocity planning. For further details, please refer to the given code comments. Please complete all TODOs for this assignment.

Once you have completed a TODO for a given function, you can uncomment it in "**module7.py**". After completing all TODOs, you should now be ready to run the simulator.

Running the CARLA simulator

In one terminal, start the CARLA simulator at a 30hz fixed time-step:

Ubuntu:

```
./CarlaUE4.sh /Game/Maps/Course4 -windowed -carla-server -benchmark -fps=30
```

Windows:

```
CarlaUE4.exe /Game/Maps/Course4 -windowed -carla-server -benchmark -fps=30
```